SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – SCENARIO BASED (CO2 – AT2): ANSWERS

Detail	Value
Name	K VENKATA VAMSI GOUD
Register No	192472021
Course Code	DSA0304
Course Name	Natural Language Processing
Topic	Morphological analysis: inflectional, derivational and rule-based parsing, stemming / normalization

Question 1: Search Engine Indexing: analyzing, analysis, analytical

Scenario Understanding

A search engine must group the word variants “analyzing”, “analysis” and “analytical” under one index entry so that a query for any variant retrieves documents containing the others.

Morphological Processing Approach

All three words share one lexical root, analyze, but the root appears in different stem variants (allomorphs): analyz-, analy- and analyt-. The program works as follows:

- Match the longest or most specific suffix rule first (-ical, -sis, -ing).
- Strip the suffix and check that the remaining stem is a known allomorph of the root.
- Map the allomorph to the root “analyze” and record the affix(es).
- Classify the suffix as inflectional (-ing: grammatical change only) or derivational (-sis, -ic, -al: new word or new word class).
- Normalize every variant to the common index key “analy”.

Python Program

```
# Q1 - Rule-based morphological processing: analyzing, analysis, analytical


# Allomorphs (stem variants) of the single lexical root "analyze"
ALLOMORPHS = {"analyz": "analyze", "analy": "analyze", "analyt": "analyze"}


# (suffix string, affixes it splits into, type, word-class change, description)
SUFFIX_RULES = [
    ("ical", ["-ic", "-al"], "Derivational", "Noun-stem -> Adjective",
     "-ic forms an adjective, -al adds 'relating to' (relating to analysis)"),
    ("sis",  ["-sis"],       "Derivational", "Verb -> Noun",
     "Greek nominal suffix: 'the process/result of analysing'"),
    ("ing",  ["-ing"],       "Inflectional", "Verb -> Verb (present participle)",
     "Inflection marks progressive aspect; word class unchanged"),
]
INDEX_KEY = "analy"          # common representation used for indexing



def parse(word):
    for suffix, affixes, kind, change, desc in SUFFIX_RULES:      # longest/most specific first
        if word.endswith(suffix):
            stem = word[: -len(suffix)]
            if stem in ALLOMORPHS:
                return {"word": word, "stem": stem, "root": ALLOMORPHS[stem],
                        "affixes": affixes, "type": kind, "change": change,
                        "note": desc, "normalized": INDEX_KEY}
    return {"word": word, "stem": word, "root": word, "affixes": [],
            "type": "None", "change": "-", "note": "no rule matched", "normalized": word}



if __name__ == "__main__":
    words = ["analyzing", "analysis", "analytical"]
    print("=" * 62)
    print("   MORPHOLOGICAL PROCESSING REPORT (SEARCH ENGINE INDEXING)")
    print("=" * 62)
    for w in words:
        r = parse(w)
        print(f"Original word    : {r['word']}")
        print(f"Stem found       : {r['stem']}-")
        print(f"Extracted root   : {r['root']}")
        print(f"Affix(es)        : {' + '.join(r['affixes'])}")
        print(f"Transformation   : {r['type']}  ({r['change']})")
```

```
        print(f"Explanation      : {r['note']}")
        print(f"Normalized output: {r['normalized']}")
        print("-" * 62)
    keys = {parse(w)["normalized"] for w in words}
    print("Unique index keys:", keys, "-> all variants grouped under one entry:",
          "YES" if len(keys) == 1 else "NO")
```

Output

```
============================================================
  MORPHOLOGICAL PROCESSING REPORT (SEARCH ENGINE INDEXING)
============================================================
Original word    : analyzing
Stem found       : analyz-
Extracted root   : analyze
Affix(es)        : -ing
Transformation   : Inflectional  (Verb -> Verb (present participle))
Explanation      : Inflection marks progressive aspect; word class unchanged
Normalized output: analy
--------------------------------------------------------------
Original word    : analysis
Stem found       : analy-
Extracted root   : analyze
Affix(es)        : -sis
Transformation   : Derivational  (Verb -> Noun)
Explanation      : Greek nominal suffix: 'the process/result of analysing'
Normalized output: analy
--------------------------------------------------------------
Original word    : analytical
Stem found       : analyt-
Extracted root   : analyze
Affix(es)        : -ic + -al
Transformation   : Derivational  (Noun-stem -> Adjective)
Explanation      : -ic forms an adjective, -al adds 'relating to' (relating to analysis)
Normalized output: analy
--------------------------------------------------------------
Unique index keys: {'analy'} -> all variants grouped under one entry: YES
```

Morphological Analysis Summary

Word	Root	Affix(es)	Type	Normalized
analyzing	analyze	-ing	Inflectional (present participle)	analy
analysis	analyze	-sis	Derivational (Verb → Noun)	analy
analytical	analyze	-ic + -al	Derivational (→ Adjective)	analy

Conclusion

The three variants reduce to one index key (analy), so a query for any form retrieves documents containing the others. This improves recall in the search engine. Only -ing is inflectional; -sis and -ical are derivational.

Question 2: Sentiment Analysis: disagree, agreement, agreeable

Scenario Understanding

A sentiment analysis platform sees reviews containing disagree, agreement and agreeable. These are formed from the same root but carry different meanings and sentiment, so the system must parse the affixes and track how they change meaning.

Morphological Processing Approach

The parser follows a prefix → suffix → root strategy:

- Strip a known prefix (dis-) if the remainder is a known root.
- Strip a known suffix (-ment, -able) if the remainder is a known root.
- Record the transformation category and its semantic effect: dis- reverses the meaning; -ment forms a noun naming a state or result; -able forms an adjective meaning capable of or inclined to.
- Apply a polarity rule: the root “agree” is positive (+1), and dis- multiplies the polarity by −1.
- Return the root “agree” as the normalized base form.

Python Program

```
# Q2 - Morphological parser: disagree, agreement, agreeable

ROOTS = {"agree": "verb"}
PREFIXES = {"dis": ("Negation / reversal of the root meaning", -1)}      # (meaning, polarity multiplier)
SUFFIXES = {
    "ment": ("Verb -> Noun", "state / result / act of the verb (agreement = a state of agreeing)", +1),
    "able": ("Verb -> Adjective", "capable of / inclined to the verb (agreeable = pleasant, willing to agree)", +1),
}
ROOT_POLARITY = {"agree": +1}      # 'agree' carries positive sentiment



def parse(word):
    prefix, suffix, rest = None, None, word
    for p in PREFIXES:                                  # step 1: prefix stripping
        if rest.startswith(p) and rest[len(p):] in ROOTS:
            prefix, rest = p, rest[len(p):]
            break
    if rest not in ROOTS:                               # step 2: suffix stripping
        for s in SUFFIXES:
            if rest.endswith(s) and rest[: -len(s)] in ROOTS:
                suffix, rest = s, rest[: -len(s)]
                break
    root = rest
    polarity = ROOT_POLARITY[root] * (PREFIXES[prefix][1] if prefix else 1)
    if prefix:
        cat, sem = "Prefixation (derivational)", PREFIXES[prefix][0] + " -> 'not agree'"
    elif suffix:
        cat, sem = f"Suffixation (derivational): {SUFFIXES[suffix][0]}", SUFFIXES[suffix][1]
    else:
        cat, sem = "Base form", "root meaning"
    return dict(word=word, prefix=prefix, root=root, suffix=suffix, cat=cat, sem=sem,
                polarity=polarity, base=root)



if __name__ == "__main__":
    print("=" * 66)
    print("   MORPHOLOGICAL PARSER - SENTIMENT ANALYSIS PIPELINE")
    print("=" * 66)
    for w in ["disagree", "agreement", "agreeable"]:
        r = parse(w)
        pol = {1: "Positive (+1)", -1: "Negative (-1)"}[r["polarity"]]
```

```
    print(f"Original word     : {r['word']}")
    print(f"Prefix            : {('dis-' if r['prefix'] else 'None')}")
    print(f"Root word         : {r['root']}")
    print(f"Suffix            : {('-' + r['suffix']) if r['suffix'] else 'None'}")
    print(f"Transformation    : {r['cat']}")
    print(f"Semantic meaning  : {r['sem']}")
    print(f"Sentiment polarity : {pol}")
    print(f"Normalized base   : {r['base']}")
    print("-" * 66)
```

Output

```
==================================================================
  MORPHOLOGICAL PARSER - SENTIMENT ANALYSIS PIPELINE
==================================================================
Original word     : disagree
Prefix            : dis-
Root word         : agree
Suffix            : None
Transformation    : Prefixation (derivational)
Semantic meaning  : Negation / reversal of the root meaning -> 'not agree'
Sentiment polarity : Negative (-1)
Normalized base   : agree
------------------------------------------------------------------
Original word     : agreement
Prefix            : None
Root word         : agree
Suffix            : -ment
Transformation    : Suffixation (derivational): Verb -> Noun
Semantic meaning  : state / result / act of the verb (agreement = a state of agreeing)
Sentiment polarity : Positive (+1)
Normalized base   : agree
------------------------------------------------------------------
Original word     : agreeable
Prefix            : None
Root word         : agree
Suffix            : -able
Transformation    : Suffixation (derivational): Verb -> Adjective
Semantic meaning  : capable of / inclined to the verb (agreeable = pleasant, willing to agree)
Sentiment polarity : Positive (+1)
Normalized base   : agree
------------------------------------------------------------------
```

Morphological Analysis Summary

Word	Prefix	Root	Suffix	Transformation	Polarity
disagree	dis-	agree	–	Prefixation (negation)	Negative (−1)
agreement	–	agree	-ment	Suffixation: Verb → Noun	Positive (+1)
agreeable	–	agree	-able	Suffixation: Verb → Adjective	Positive (+1)

Conclusion

All three forms share the base “agree”, but the derivational affix decides the sentiment: the prefix dis- flips polarity, while the suffixes -ment and -able keep it positive. A sentiment system therefore needs the affix information as well as the root to interpret each word correctly.

Question 3: Topic Modeling: govern, government, governance

Scenario Understanding

A news aggregation pipeline treats govern, government and governance as three separate tokens, which splits the term frequency of one concept across three features and weakens topic modeling and clustering.

Morphological Processing Approach

The module decomposes each word into the root and successive derivational suffix layers:

- Repeatedly strip the known derivational suffixes (-ment, -ance) until a known root remains.
- Count the layers to get the derivational level: govern = level 0, government = level 1, governance = level 1.
- Build the hierarchy, for example govern (V) → government (N).
- Map every variant to the common lexical form “govern”, which is the final token passed to topic modeling or clustering.

Python Program

```
# Q3 - Morphology-based normalization: govern, government, governance
from collections import Counter

ROOTS = {"govern": "Verb"}
SUFFIXES = {   # suffix: (resulting class, meaning)
   "ment": ("Noun", "result / system / body that governs"),
   "ance": ("Noun", "act, state or quality of governing"),
}



def decompose(word):
   """Return root + list of derivational suffix layers (innermost first)."""
   layers, w = [], word
   while w not in ROOTS:
      for s in SUFFIXES:
         if w.endswith(s):
            layers.insert(0, s)
            w = w[: -len(s)]
            break
      else:
         return None
   return w, layers



def analyse(word):
   root, layers = decompose(word)
   level = len(layers)
   chain = [root]
   for s in layers:
      chain.append(chain[-1] + s)
   hierarchy = " -> ".join(f"{c}({ROOTS[root][0] if i == 0 else SUFFIXES[layers[i-1]][0][0]})"
                           for i, c in enumerate(chain))
   return dict(word=word, root=root, affixes=[f"-{s}" for s in layers], level=level,
               hierarchy=hierarchy, normalized=root, final=root)



if __name__ == "__main__":
   words = ["govern", "government", "governance"]
   print("=" * 68)
   print("   MORPHOLOGY-BASED NORMALIZATION (TOPIC MODELING / CLUSTERING)")
   print("=" * 68)
   for w in words:
```

```
    r = analyse(w)
    print(f"Original word         : {r['word']}")
    print(f"Root form             : {r['root']}")
    print(f"Detected affix(es)    : {', '.join(r['affixes']) or 'None'}")
    print(f"Derivational level    : {r['level']}")
    print(f"Derivational hierarchy : {r['hierarchy']}")
    print(f"Normalized form       : {r['normalized']}")
    print(f"Final token for model  : {r['final']}")
    print("-" * 68)
# Demonstration: effect on term frequencies
doc = "the government must govern well and governance reflects how leaders govern".split()
raw = Counter(w for w in doc if w.startswith("govern"))
norm = Counter(analyse(w)["final"] for w in doc if w.startswith("govern"))
print("Term counts before normalization:", dict(raw))
print("Term counts after  normalization:", dict(norm))
```

Output

```
====================================================================
  MORPHOLOGY-BASED NORMALIZATION (TOPIC MODELING / CLUSTERING)
====================================================================
Original word         : govern
Root form             : govern
Detected affix(es)    : None
Derivational level    : 0
Derivational hierarchy : govern(V)
Normalized form       : govern
Final token for model  : govern
--------------------------------------------------------------------
Original word         : government
Root form             : govern
Detected affix(es)    : -ment
Derivational level    : 1
Derivational hierarchy : govern(V) -> government(N)
Normalized form       : govern
Final token for model  : govern
--------------------------------------------------------------------
Original word         : governance
Root form             : govern
Detected affix(es)    : -ance
Derivational level    : 1
Derivational hierarchy : govern(V) -> governance(N)
Normalized form       : govern
Final token for model  : govern
--------------------------------------------------------------------
Term counts before normalization: {'government': 1, 'govern': 2, 'governance': 1}
Term counts after  normalization: {'govern': 4}
```

Morphological Analysis Summary

Word	Root	Affix	Level	Hierarchy	Normalized
govern	govern	–	0	govern(V)	govern
government	govern	-ment	1	govern(V) → government(N)	govern
governance	govern	-ance	1	govern(V) → governance(N)	govern

Conclusion

-ment and -ance are both Verb → Noun derivational suffixes at level 1, so each word has the same root. In the sample document the separate counts (govern 2, government 1, governance 1) merge into a single feature, govern = 4, which gives stronger and cleaner features for topic modeling and clustering.

Question 4: Document Classification: activate, activation, reactivation

Scenario Understanding

A document classifier must group activate, activation and reactivation. They differ by stacked derivational layers, not just one affix, so the parser must identify every layer and its effect on word class and meaning.

Morphological Processing Approach

The parser removes the prefix, then peels suffixes layer by layer, using a lexicon check and a spelling rule (e-restoration: activ + e → active):

- Strip the prefix re- if the remainder is a known word.
- Peel suffixes in turn: -ion (activation → activate), -ate (activate → active), -ive (active → act).
- Stop at the root "act" and reverse the steps to give the derivational sequence.
- Record the effect of each step on word class and meaning: act (V) → active (Adj) → activate (V) → activation (N) → reactivation (N).
- Map all three words to the normalized base "activate".

Python Program

```
# Q4 - Morphological parsing: activate, activation, reactivation

LEXICON = {"act": "Verb/Noun", "active": "Adjective", "activate": "Verb",
           "activation": "Noun"}
ROOTS = {"act"}
PREFIXES = {"re": "again / back (repetition)"}
SUFFIXES = {"ion": ("Noun", "process / result of the action"),
            "ate": ("Verb", "to make / cause to be"),
            "ive": ("Adjective", "having the quality of")}



def peel(word):
    """Strip prefix, then repeatedly strip suffixes (with spelling restoration)."""
    prefix, w = None, word
    for p in PREFIXES:
        if w.startswith(p) and w[len(p):] in LEXICON:
            prefix, w = p, w[len(p):]
            break
    steps = []                              # (suffix, from_word, to_word)
    while w not in ROOTS:
        for s in SUFFIXES:
            if w.endswith(s):
                stem = w[: -len(s)]
                cand = next((c for c in (stem, stem + "e") if c in LEXICON), None)
                if cand:                    # e-restoration: activ+e -> active
                    steps.append((s, cand, w))
                    w = cand
                    break
        else:
            break
    return prefix, w, list(reversed(steps))      # innermost step first



def analyse(word):
    prefix, root, steps = peel(word)
    seq = [f"{root} ({LEXICON[root].split('/')[0]})"]
    effects = []
    for s, frm, to in steps:
        cls, meaning = SUFFIXES[s]
        seq.append(f"+{s} -> {to} ({cls})")
```

```
        effects.append(f"{frm} -> {to}: {LEXICON[frm].split('/')[0]} -> {cls}; {meaning}")
    if prefix:
        seq.append(f"re- + {word[2:]} -> {word} (Noun)")
        effects.append(f"{word[2:]} -> {word}: class unchanged (Noun); meaning '{PREFIXES[prefix]}'")
    return dict(word=word, prefix=prefix, root=root, suffixes=[s for s, _, _ in steps],
                seq=seq, effects=effects, base="activate",
                parsed=f"{('re-' if prefix else '')}[{root}]" + "".join(f"-{s}" for s, _, _ in steps))



if __name__ == "__main__":
    print("=" * 70)
    print("   MORPHOLOGICAL PARSING & NORMALIZATION - DOCUMENT CLASSIFICATION")
    print("=" * 70)
    for w in ["activate", "activation", "reactivation"]:
        r = analyse(w)
        print(f"Original word        : {r['word']}")
        print(f"Prefix               : {('re-' if r['prefix'] else 'None')}")
        print(f"Root word            : {r['root']}")
        print(f"Suffix(es)           : {', '.join('-' + s for s in r['suffixes']) or 'None'}")
        print("Derivational sequence :")
        for line in r["seq"]:
            print("      ", line)
        print("Effect on class/meaning:")
        for line in r["effects"]:
            print("      ", line)
        print(f"Normalized base form  : {r['base']}")
        print(f"Final parsed form     : {r['parsed']}")
        print("-" * 70)
```

Output

```
======================================================================
   MORPHOLOGICAL PARSING & NORMALIZATION - DOCUMENT CLASSIFICATION
======================================================================
Original word        : activate
Prefix               : None
Root word            : act
Suffix(es)           : -ive, -ate
Derivational sequence :
      act (Verb)
      +ive -> active (Adjective)
      +ate -> activate (Verb)
Effect on class/meaning:
      act -> active: Verb -> Adjective; having the quality of
      active -> activate: Adjective -> Verb; to make / cause to be
Normalized base form  : activate
Final parsed form     : [act]-ive-ate
----------------------------------------------------------------------
Original word        : activation
Prefix               : None
Root word            : act
Suffix(es)           : -ive, -ate, -ion
Derivational sequence :
      act (Verb)
      +ive -> active (Adjective)
      +ate -> activate (Verb)
      +ion -> activation (Noun)
Effect on class/meaning:
      act -> active: Verb -> Adjective; having the quality of
      active -> activate: Adjective -> Verb; to make / cause to be
      activate -> activation: Verb -> Noun; process / result of the action
Normalized base form  : activate
Final parsed form     : [act]-ive-ate-ion
----------------------------------------------------------------------
Original word        : reactivation
Prefix               : re-
```

```
Root word            : act
Suffix(es)           : -ive, -ate, -ion
Derivational sequence :
      act (Verb)
      +ive -> active (Adjective)
      +ate -> activate (Verb)
      +ion -> activation (Noun)
      re- + activation -> reactivation (Noun)
Effect on class/meaning:
      act -> active: Verb -> Adjective; having the quality of
      active -> activate: Adjective -> Verb; to make / cause to be
      activate -> activation: Verb -> Noun; process / result of the action
      activation -> reactivation: class unchanged (Noun); meaning 'again / back (repetition)'
Normalized base form  : activate
Final parsed form     : re-[act]-ive-ate-ion
------------------------------------------------------------------------
```

Morphological Analysis Summary

Word	Prefix	Root	Suffixes	Derivational sequence
activate	–	act	-ive, -ate	act(V) → active(Adj) → activate(V)
activation	–	act	-ive, -ate, -ion	act(V) → active(Adj) → activate(V) → activation(N)
reactivation	re-	act	-ive, -ate, -ion	… activation(N) → re- + activation = reactivation(N)

Conclusion

The root is “act” for all three words. The suffixes change the word class at each step (V → Adj → V → N). The prefix re- changes only the meaning (“again”), not the class. Normalizing to “activate” groups all three words under one concept for classification and semantic indexing.

Question 5: Search Optimization: create, creates, creating

Scenario Understanding

A search module must return the same results for create, creates and creating. These are inflectional variants only, so the normalization must not change the meaning of the word.

Morphological Processing Approach

The module applies inflectional suffix rules with e-restoration:

- If the word is already in the lexicon it is the base form (VB).
- Otherwise match the suffix: -s → third-person singular present (VBZ), -es → the same category, -ing → present participle (VBG).
- Strip the suffix and restore the dropped "e" when needed (creat + ing → creat + e = create).
- Return the root "create" as the normalized base form and index key for every variant.

Python Program

```
# Q5 - Inflectional morphology normalization: create, creates, creating

LEXICON = {"create"}                    # known base forms (lemmas)
# (suffix, grammatical category, Penn Treebank tag, feature)
RULES = [
    ("ing", "Present participle", "VBG", "progressive aspect"),
    ("s",   "3rd-person singular present", "VBZ", "singular subject, present tense"),
    ("es",  "3rd-person singular present", "VBZ", "singular subject, present tense"),
]



def normalize(word):
    if word in LEXICON:
        return dict(word=word, suffix="-", cat="Base form", tag="VB",
                    feature="infinitive / plural or 1st-2nd person present",
                    root=word, base=word)
    for suf, cat, tag, feat in RULES:
        if word.endswith(suf):
            stem = word[: -len(suf)]
            for cand in (stem, stem + "e"):        # e-restoration: creat + e
                if cand in LEXICON:
                    return dict(word=word, suffix="-" + suf, cat=cat, tag=tag,
                                feature=feat, root=cand, base=cand)
    return dict(word=word, suffix="?", cat="Unknown", tag="?", feature="-", root=word, base=word)



if __name__ == "__main__":
    print("=" * 66)
    print("   INFLECTIONAL NORMALIZATION - SEARCH OPTIMIZATION MODULE")
    print("=" * 66)
    results = [normalize(w) for w in ["create", "creates", "creating"]]
    for r in results:
        print(f"Original word      : {r['word']}")
        print(f"Identified suffix  : {r['suffix']}")
        print(f"Grammatical category: {r['cat']} [{r['tag']}] - {r['feature']}")
        print(f"Extracted root     : {r['root']}")
        print(f"Normalized base form: {r['base']}")
        print(f"Final representation: {r['base']}  (index key)")
        print("-" * 66)
    print("All variants map to one key:", {r['base'] for r in results})
```

Output

```
================================================================
  INFLECTIONAL NORMALIZATION - SEARCH OPTIMIZATION MODULE
================================================================
Original word      : create
Identified suffix  : -
Grammatical category: Base form [VB] - infinitive / plural or 1st-2nd person present
Extracted root     : create
Normalized base form: create
Final representation: create  (index key)
----------------------------------------------------------------
Original word      : creates
Identified suffix  : -s
Grammatical category: 3rd-person singular present [VBZ] - singular subject, present tense
Extracted root     : create
Normalized base form: create
Final representation: create  (index key)
----------------------------------------------------------------
Original word      : creating
Identified suffix  : -ing
Grammatical category: Present participle [VBG] - progressive aspect
Extracted root     : create
Normalized base form: create
Final representation: create  (index key)
----------------------------------------------------------------
All variants map to one key: {'create'}
```

Morphological Analysis Summary

Word	Suffix	Grammatical category	Tag	Root / Normalized
create	–	Base form	VB	create
creates	-s	3rd-person singular present	VBZ	create
creating	-ing	Present participle (progressive)	VBG	create

Conclusion

All three words are inflectional forms of one lemma, “create”, and inflection changes only the grammatical feature (tense, aspect or agreement), not the meaning or word class. Mapping them to one key gives consistent retrieval and better search recall.